

Machine Learning for Natural Language Understanding

Prof. Achim Rettinger – Master NLP

Sources

- Multilingual
 - Language-specific LLM for Low-Resource Languages
 - <https://pieter.ai/trans-tokenization/>
 - European Languages
 - <https://opengpt-x.de/en/models/teuken-7b/>
 - Massively Multilingual
 - <https://aclanthology.org/2025.coling-main.428/>
- Multimodal
 - <https://mistral.ai/news/pixtral-12b/>
- Multitool
 - https://reports.weforum.org/docs/WEF_AI_Agents_in_Action_Foundations_for_Evaluation_and_Governance_2025.pdf
 - <https://modelcontextprotocol.io/docs/getting-started/intro>
- Multiagent
 - <https://www.byhand.ai/p/llm-vs-rag-vs-agent-workbook>
 - <https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/>

Flipped Classroom Questions

- Name technical challenges of multilingual LLMs
- What core component needs to be added to make an LLM multimodal?
- How does an MCP-Client find out if there is a suitable tool provided by an MCP-Server?
- What is the difference wrt inference of “LLMs”, “RAG”, “Tool Use” and “Multi agent” with respect to their control logic?

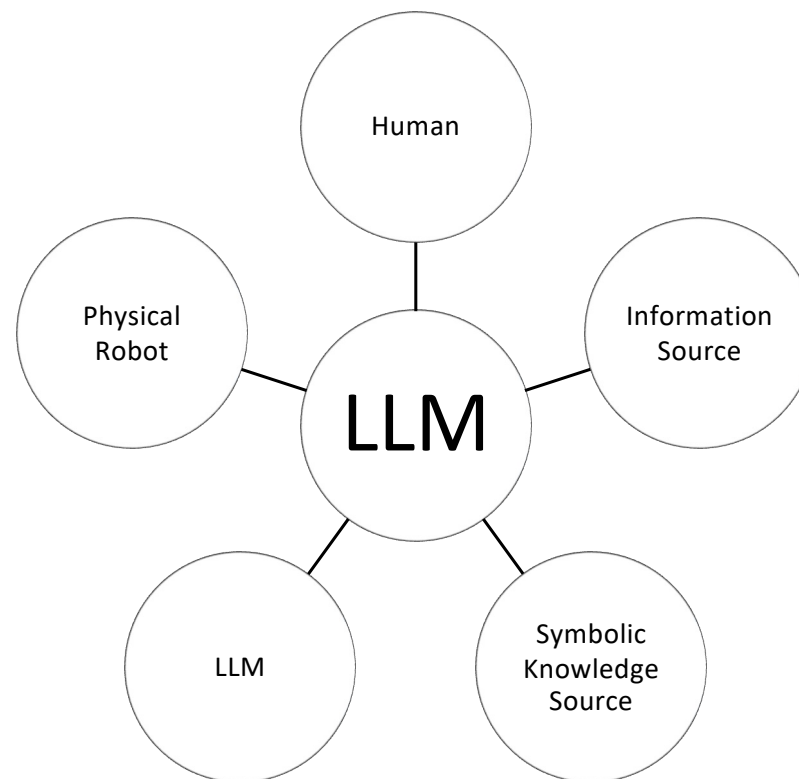


Multi -Lingual / -Modal / -Tool / -Agent

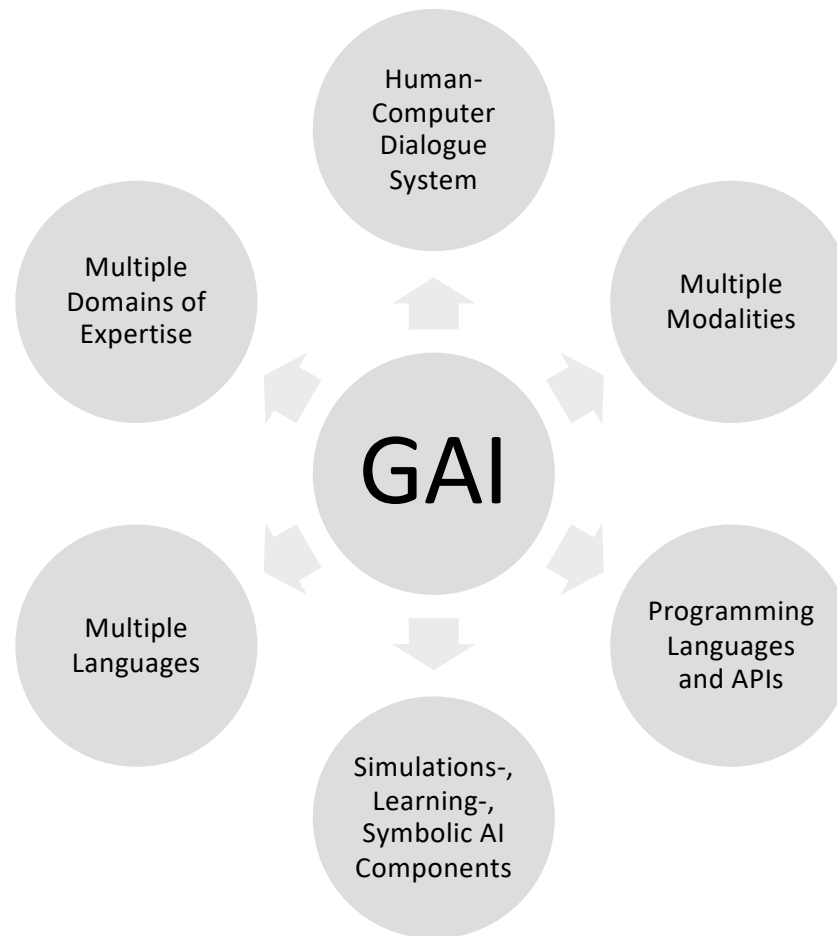
Lecture 11

Recap: LLMs are translator by nature

Their underlying neural network architecture is “Transformers”



Recap: LLMs can be “the glue” of hybrid systems



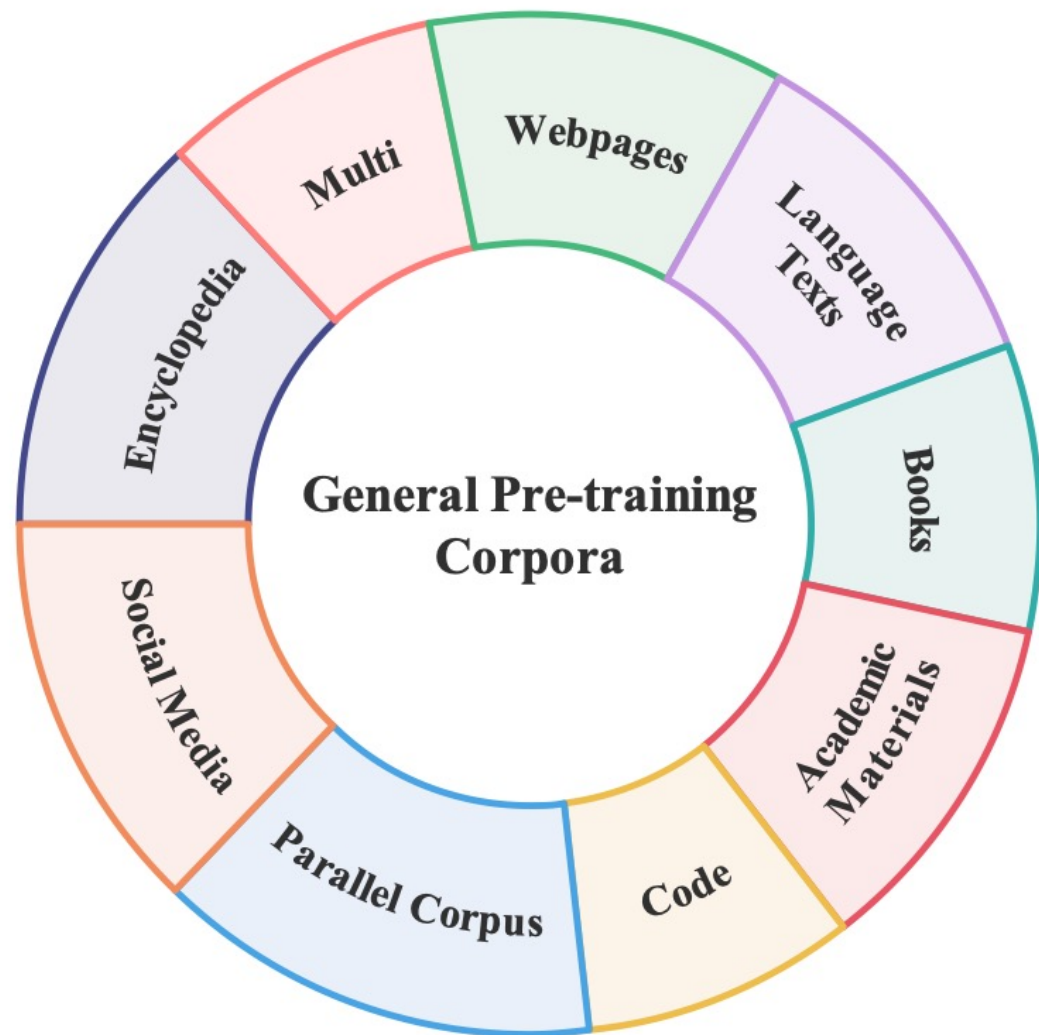
Multi-Lingual LLMs

Section 11.1

Recap: Types of Data of General Pre-training Corpora (<https://arxiv.org/pdf/2402.18041>)

Largest Source:
<https://commoncrawl.org>

LLama3: “Final data mix contains roughly 50% of tokens corresponding to general knowledge, 25% of mathematical and reasoning tokens, 17% code tokens, and 8% multilingual tokens.”



Language-specific Model for Lower-Resource Languages

Dutch: <https://arxiv.org/pdf/2408.04303>

German: <https://huggingface.co/flair/bueble-lm-2b>

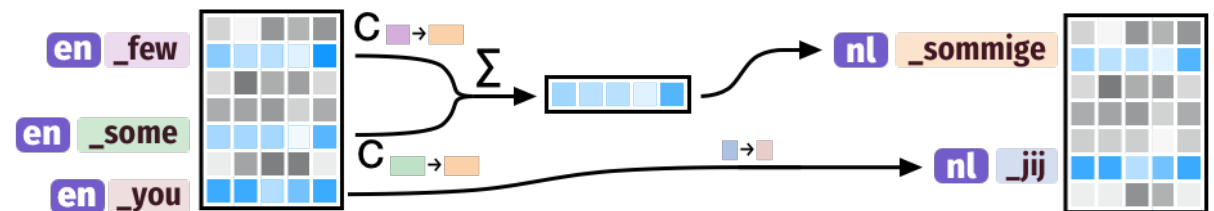
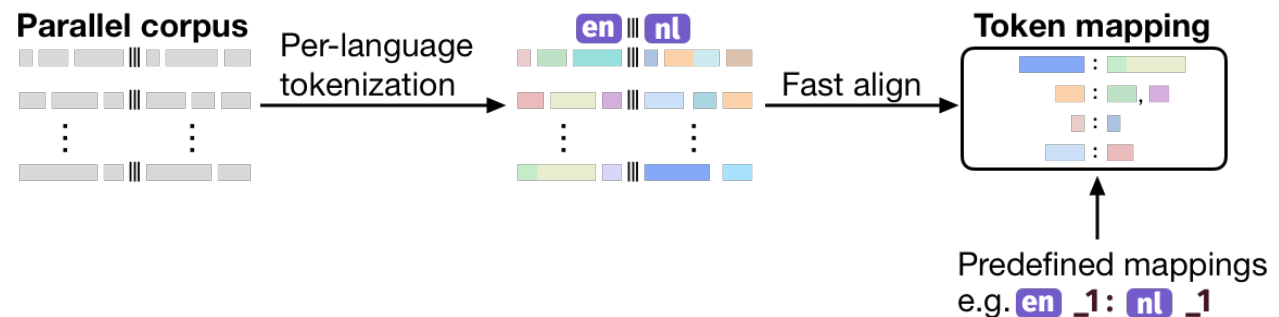
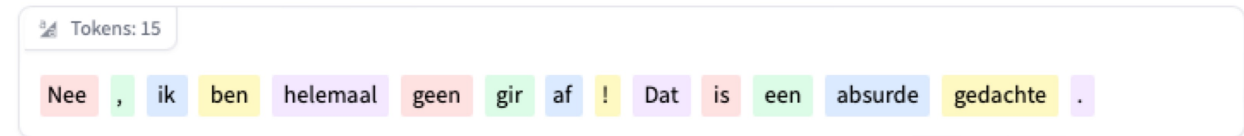
Issues with multilingual models:

- multilingual tokenization not optimal

Issues with lower-resource languages:

- not enough training data to train from scratch

One idea: Trans-tokenization



Teuken Models

<https://arxiv.org/pdf/2410.03730>

- Training from scratch on 24 European Languages only
- 60% non-English data
- Post Training mostly on English and German

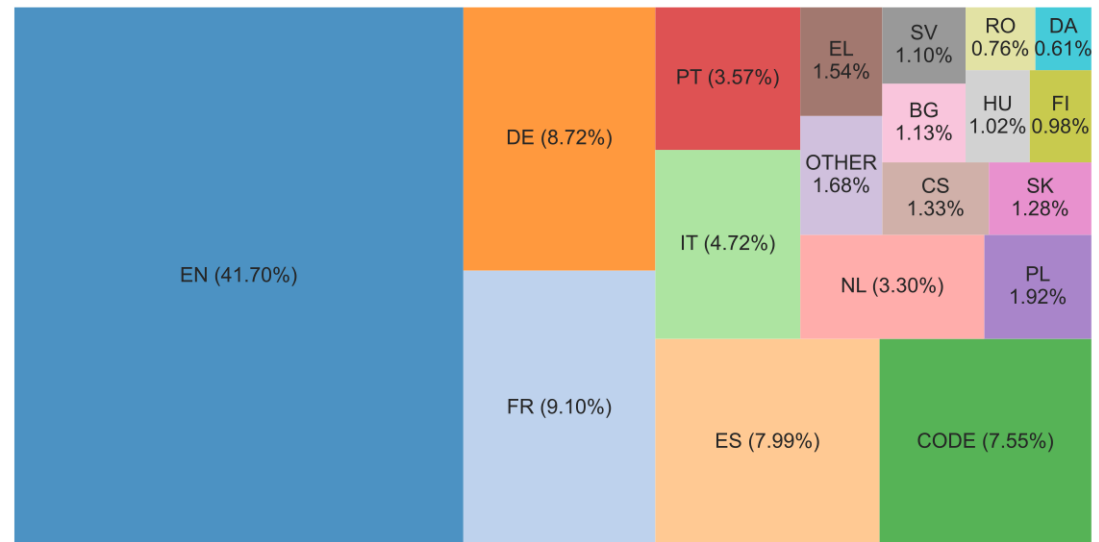
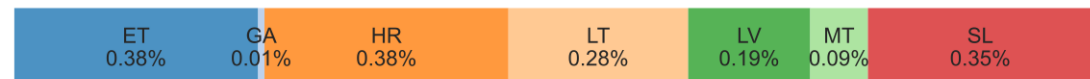


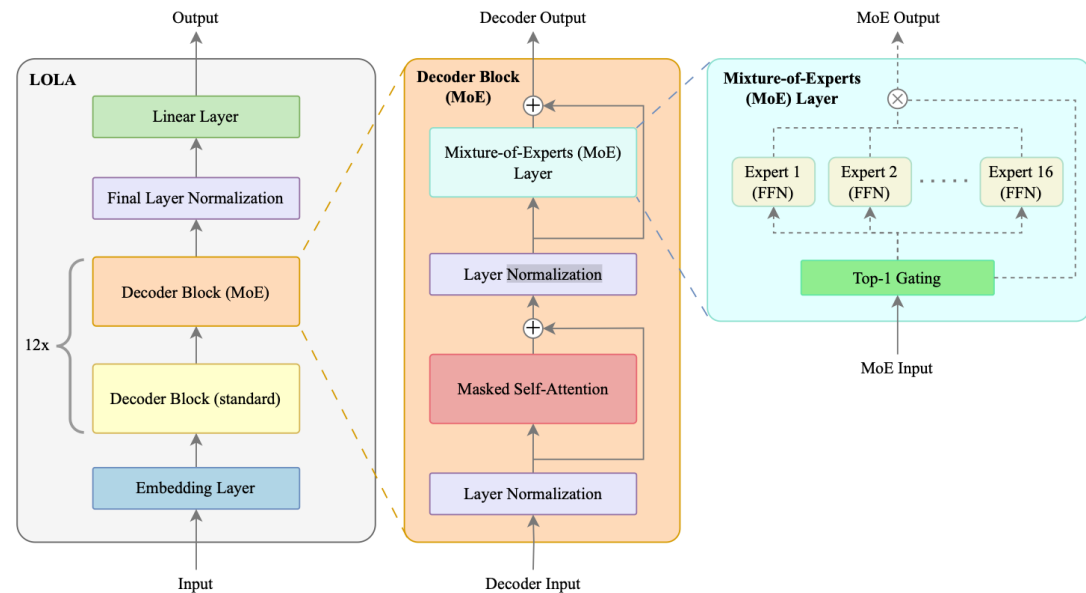
Figure 1: Language distribution of the tokenized dataset, comparing the presence of English and other European languages.



LOLA Model

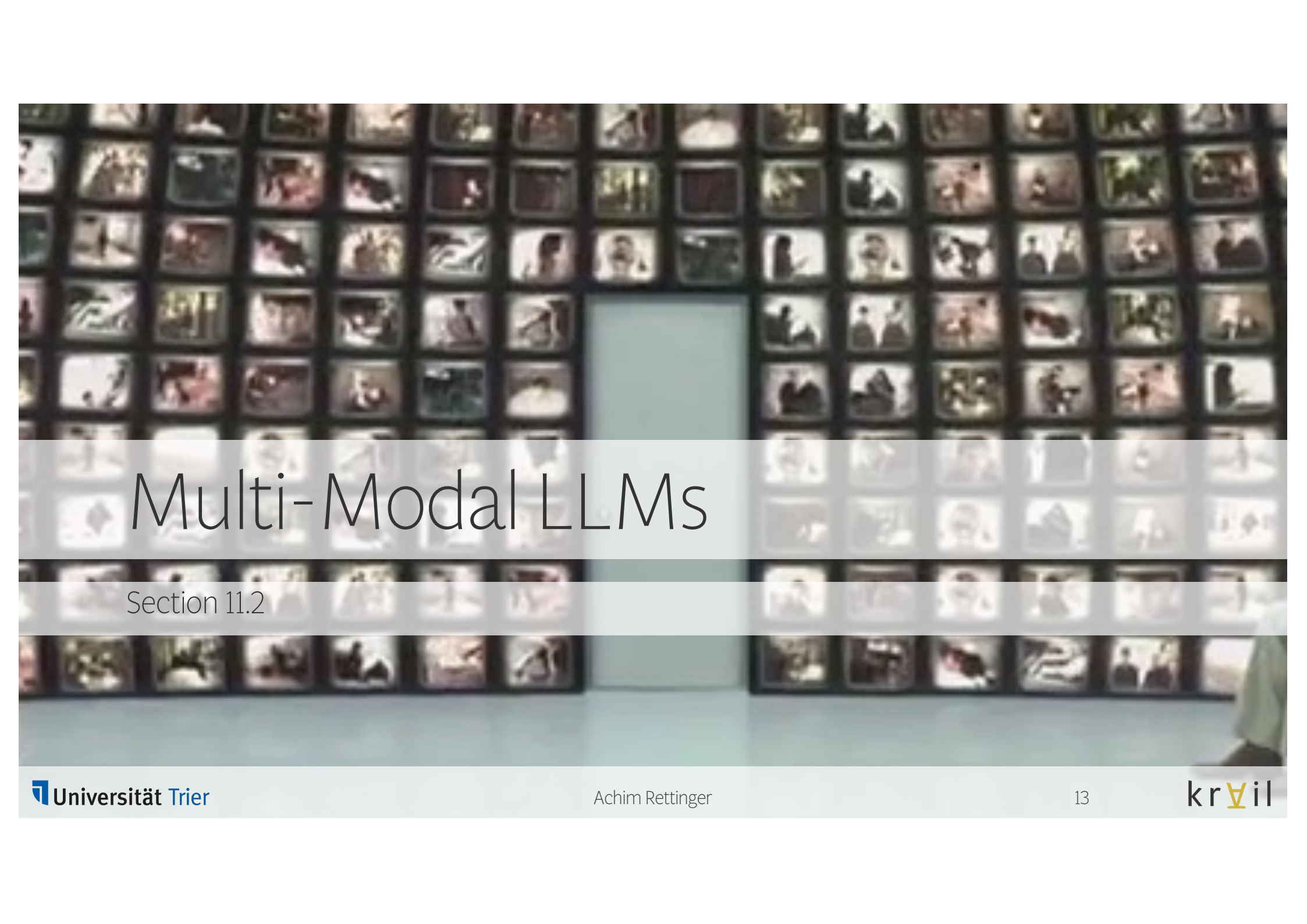
<https://aclanthology.org/2025.coling-main.428/>

- Curse of multilinguality: the ability of models to generalize across multiple languages diminishes unless their capacity is significantly expanded
- Idea: Mixture of Experts (replace FFN-layer with MoEs)
- Trained on 160 languages



LLMs can go beyond human languages

- Morse code
 - Genetic code
 - Hieroglyphics
 - Sign language
 - Musical notation
 - Chemical signaling
 - ...
- Emojis and symbols
 - Animal communication
 - Haptic communications
 - Traffic signs and signals
 - Mathematical equations
 - Programming languages
 - ...



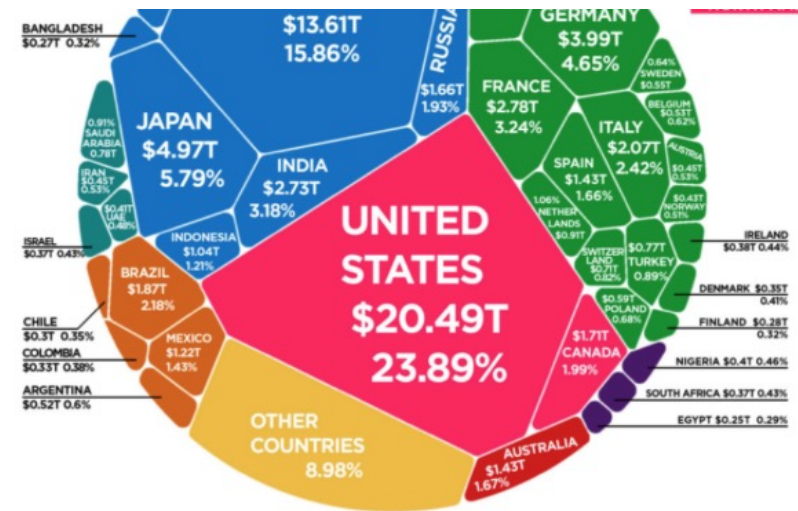
Multi-Modal LLMs

Section 11.2

Examples of Multimodal Tasks

Reasoning over complex figures

<https://arxiv.org/pdf/2410.07073>



Prompt:

List the top 5 countries in Europe with the highest GDP

Response:

The top 5 countries in Europe with the highest GDP, based on the image, are:

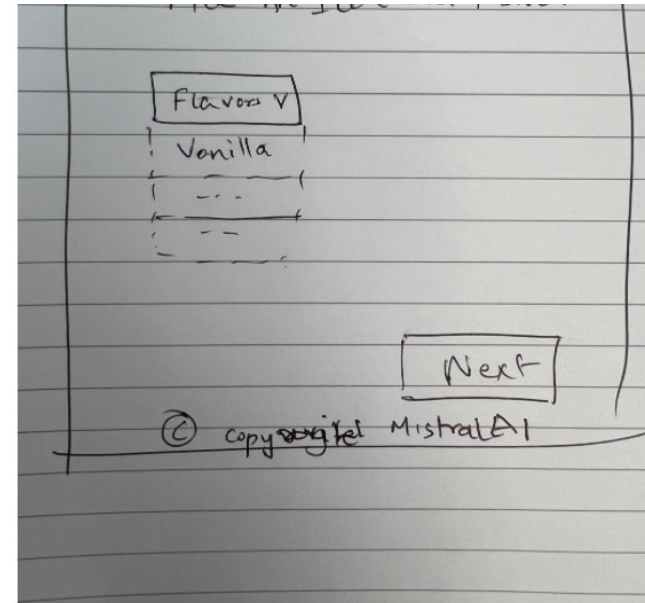
1. **Germany**
 - GDP: \$3.99 trillion
 - Percentage: 4.65%
2. **United Kingdom**
 - GDP: \$2.82 trillion
 - Percentage: 3.29%
3. **France**
 - GDP: \$2.78 trillion
 - Percentage: 3.24%
4. **Italy**

Achim Rettinger

Examples of Multimodal Tasks

Image to Code

<https://arxiv.org/pdf/2410.07073>



Prompt:

Write HTML code to create a website like this

Response:

Certainly! Below is an example of HTML code to create a simple webpage that allows users to pick an ice cream flavor. This example includes a dropdown menu for selecting the flavor and a "Next" button.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Pick an Ice Cream Flavor</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      display: flex;
      justify-content: center;
      align-items: center;
      height: 100vh;
      background-color: #f0f0f0;
    }
  </style>
</head>
<body>
  <div>
    <div>
      <div>Flavor V</div>
      <div>
        <div>Vanilla</div>
        <div></div>
        <div></div>
      </div>
      <div>Next</div>
    </div>
    <div>© copyrigtel Mistrat AI</div>
  </div>
</body>
</html>
```

Examples of Multimodal Tasks: Natural scene understanding

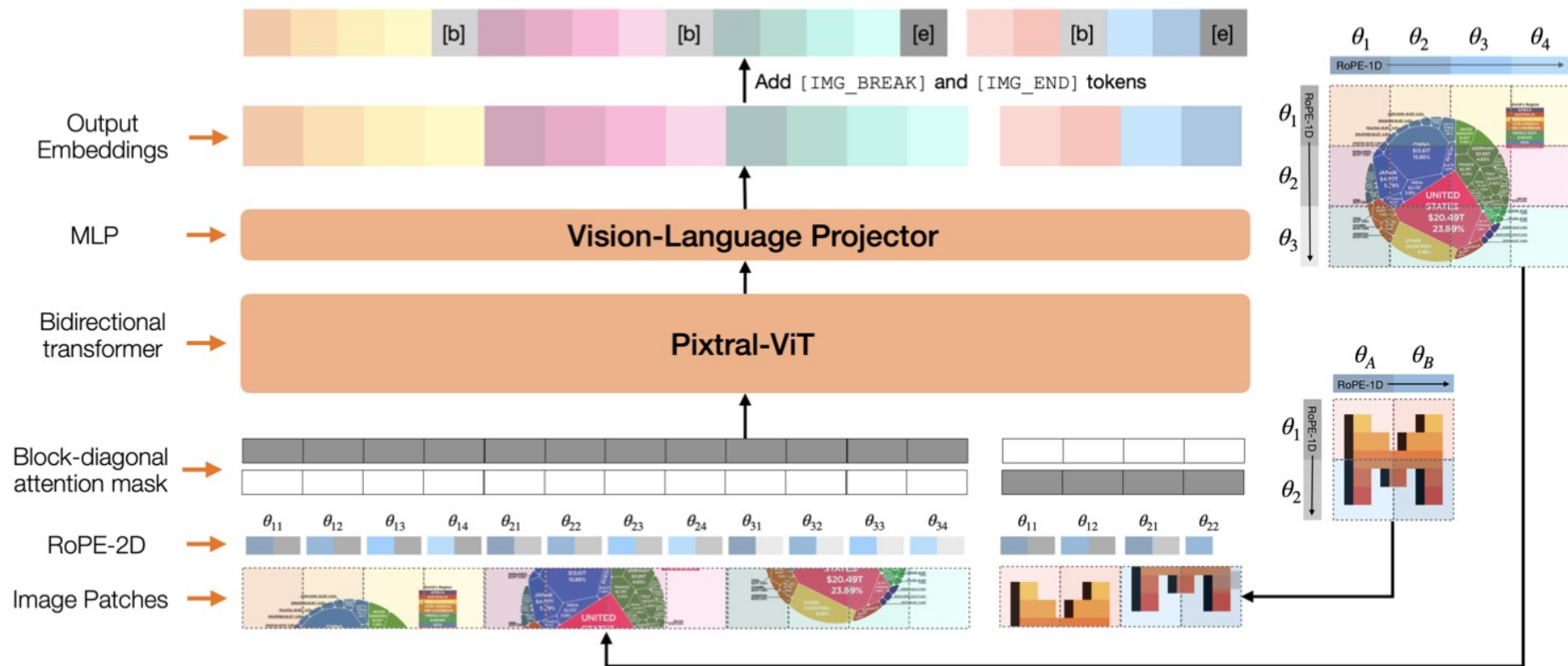


Prompt: Is this person really big, or is this building just super small?

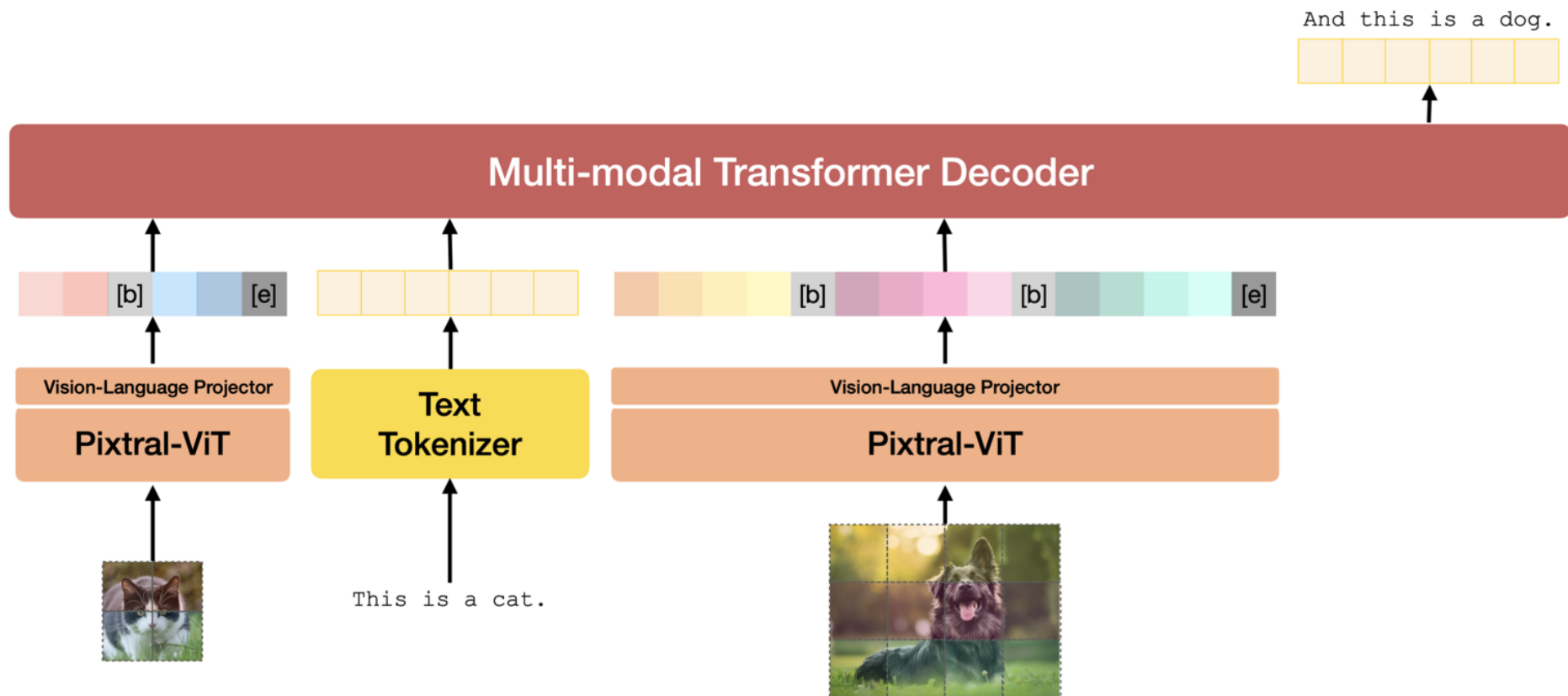
Response:

The image shows a person holding their thumb in front of the Leaning Tower of Pisa, making it appear as if the thumb is much larger than the tower. This creates an optical illusion, making the tower seem smaller than it actually is. In reality, the Leaning Tower of Pisa is a large and iconic structure, and the person's thumb is not nearly as large as it appears in the image. The purpose of this photo is to demonstrate the impressive height and size of the tower by comparing it to a familiar object, such as a thumb.

Pixtral Vision Encoder <https://arxiv.org/pdf/2410.07073>



Complete Pixtral Architecture <https://arxiv.org/pdf/2410.07073>





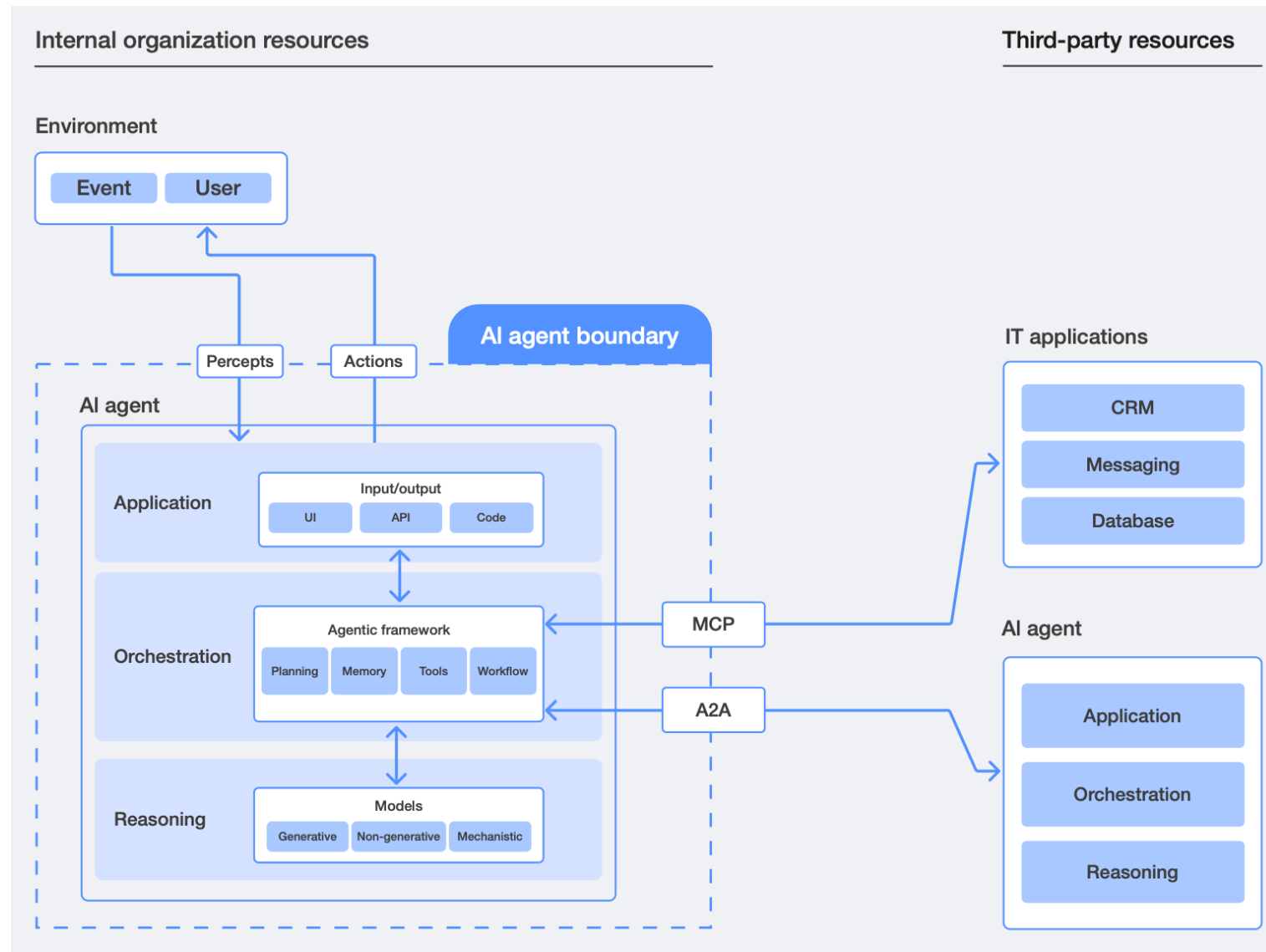
Multi-Tool LLMs

Lecture 11.3

Software-architectures of AI Agents

Model Context Protocol (MCP) vs Agent-to-Agent (A2A):

Where MCP focuses on communication between agents and external or internal systems, protocols like A2A enable agents to discover each other, interact, collaborate and delegate tasks



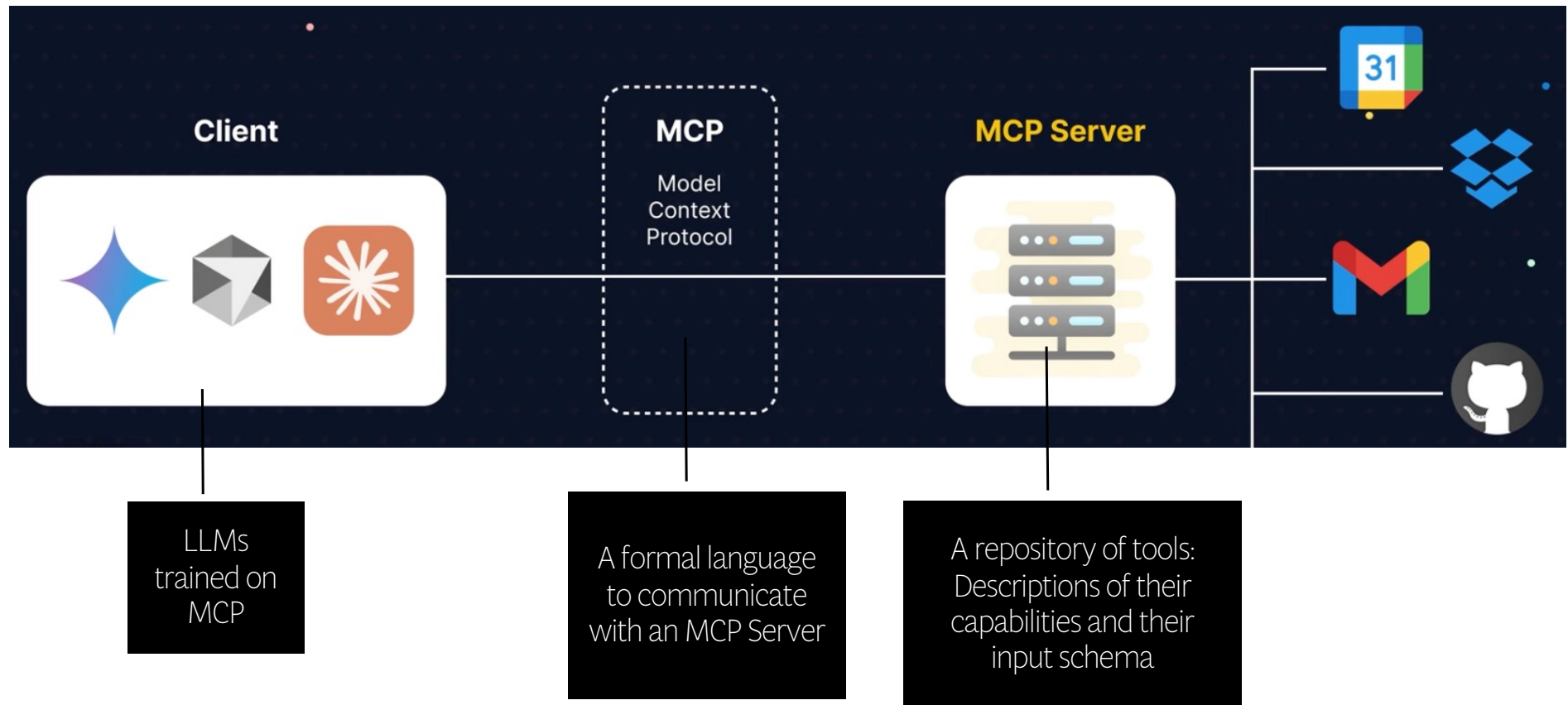
https://reports.weforum.org/docs/WEF_AI_Agents_in_Action_Foundations_for_Evaluation_and_Governance_2025.pdf

Achim Rettinger

20

What is MCP?

<https://youtu.be/bC3mlQWHZMQ?si=WkgFgJKMXQw9Hx60>



Example of Tool Description

```
<> JSON
{
  "name": "calculate_sum",
  "description": "Adds two numbers together.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "a": {
        "type": "number",
        "description": "The first number to add"
      },
      "b": {
        "type": "number",
        "description": "The second number to add"
      }
    },
    "required": ["a", "b"]
  }
}
```

Client (LLM) <-> MCP Server Interaction

1. Discovery: The client asks the MCP server for a list of tools.
2. Selection: The LLM analyzes the description fields of available tools to see if any match the user's request.
3. Invocation: If the LLM decides to use a tool, it generates arguments based on the inputSchema.
4. Execution: The MCP server receives the request, executes the underlying API logic (the actual Python/TypeScript code), and returns the result to the model.



Multi-Agent LLMs

Section 11.4

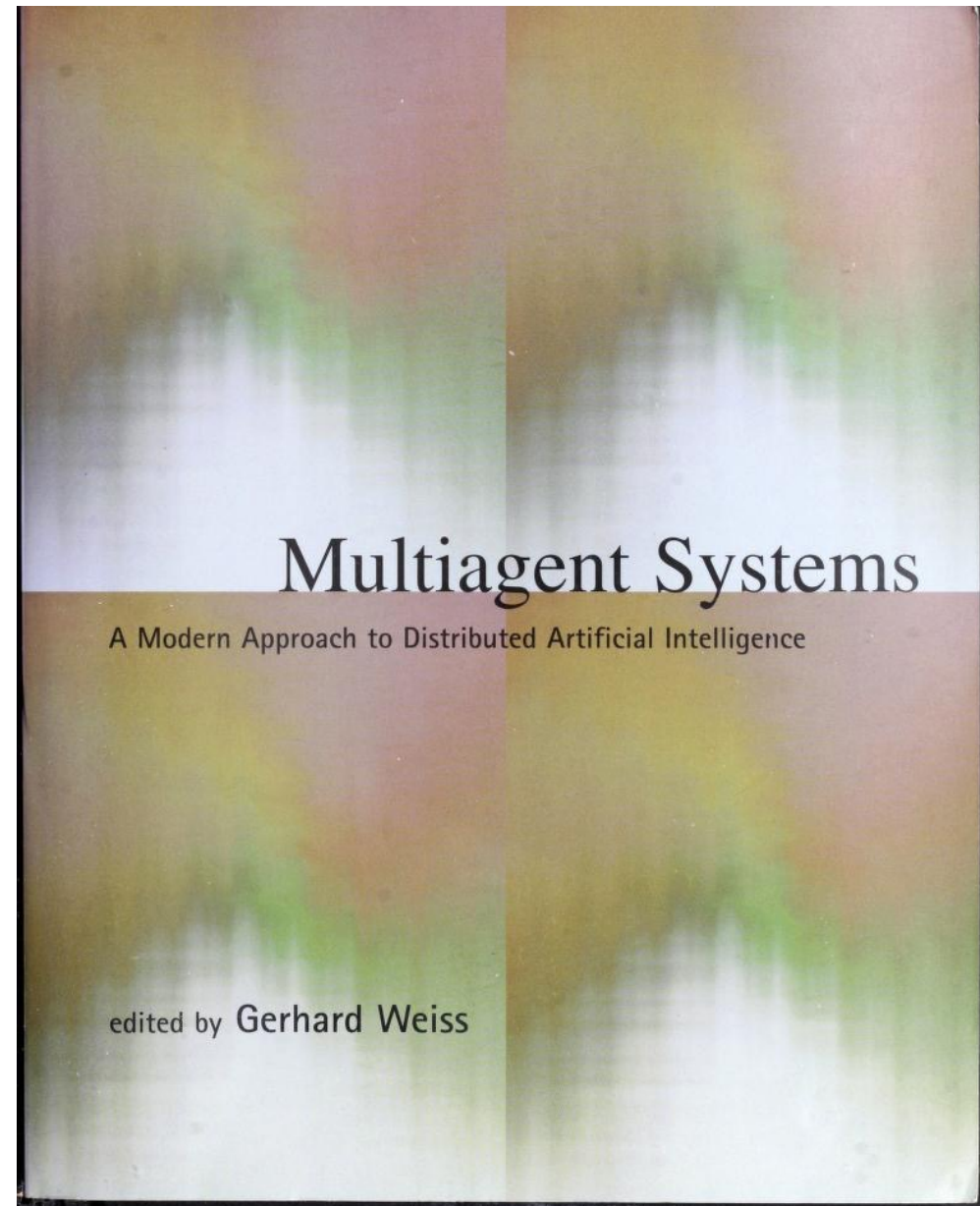
Some history lesson

Interacting Intelligent Agents:

- "Agents" are autonomous, computational entities that can be viewed as perceiving their environment through sensors and acting upon their environment through effectors.
- "Intelligent" indicates that the agents pursue their goals and execute their tasks such that they optimize some given performance measures. To say that agents are intelligent does not mean that they are omniscient or omnipotent, nor does it mean that they never fail.
- "Interacting" indicates that the agents may be affected by other agents or perhaps by humans in pursuing their goals and executing their tasks. Interaction can take place indirectly through the environment in which they are embedded or directly through a shared language.

Multiagent Systems:

- each agent has just incomplete information and is restricted in its capabilities;
- system control is distributed;
- data is decentralized; and
- computation is asynchronous.



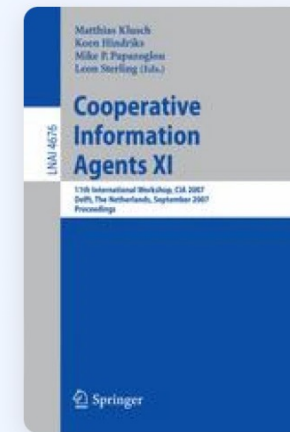
Personal history (about 20 year ago)

[Home](#) > [Cooperative Information Agents XI](#) > Conference paper

Learning Initial Trust Among Interacting Agents

Conference paper

pp 313–327 | [Cite this conference paper](#)



Cooperative Information Agents XI
(CIA 2007)

Achim Rettinger, Matthias Nickles & Volker Tresp

Timeline, simplified

- ~1950s–1960s: Early AI and agent-related concepts. First problem-solving and decision-making systems
- ~1970s–1990s: High hopes for Symbolic AI. Thinking about intelligent agents. Two AI winters.
- ~1980s: Distributed AI and early Multi-Agent Systems
- ~1990s–2000s: The heyday and decline of old-school agents. Rise of Machine Learning
- ~2010s: A new AI spring. Deep Learning matures. Generative AI starts taking over the AI world
- ~2020s (so far): GenAI at scale. The great Agentic AI and Reinforcement Learning (RL) revival

What is an AI agent?

- Various definitions exist in the AI community
- “Intelligent agent” may refer to all kinds of (more or less) intelligent entities, including humans, many animals, agentic AI...
- A simple definition often used AI: an artificial intelligent entity which
 - perceives its environment
 - proactively takes actions towards an objective (e.g., goal, desire)

What is "Agentic AI"?

- Agentic AI: Technically the same as "AI Agent"
- However, "Agentic AI" usually refers to AI Agents based on Large Language Models (LLMs)
- Many current AI agents (and most current AI apps) are essentially LLM-wrappers with function/API calling capability
- "Agentic AI" is rather imprecise => "GenAI agents" might be better

Some AI Agent Models throughout history

- Reflex agents act based on current percept
- Reactive agents act based solely on the current situation
- Model-based agents maintain a model of the world
- Logic-based agents use logical reasoning (deductive/inductive/abductive)
- BDI-agents make decision based on beliefs, desires and intentions
- Rational agents act rationally towards a goal
- Goal-based agents act towards a goal
- Utility-based agents select actions based on the utility of states they are in
- Learning agents increase their knowledge about the environment
- Robots: Agents with hardware for acting in a physical environment
- Multimodal agents powered by vision models
- Autonomous agents perform complex tasks (more or less) independently
- Distributed agents are systems of agents running on multiple machines
- Collaborative agents work together towards a goal
- A Multi-Agent System (MAS): a system of multiple interacting intelligent agents
- Combinations exist (e.g., goal-based learning agents)
- Different authors use different, often overlapping terms (e.g., “reflex agent”, “reactive agent”...)

Interacting Agents (old vs new)

- Multi-Agent Systems (MAS) consist of multiple interacting agents
- No individual MAS-agent has full control or full overview of the environment
- Manual design or self-organisation to various degrees
- Prominent approach to automated task allocation in old-school MAS:
 - Contract Net Protocol (CNP)
- Old-school MAS interaction often modelled on linguistic or sociological approaches to agent communication, but typically not using natural language
- Implementation focussed on Agent-Communication Languages and Interaction frameworks, e.g.
 - DARPA's Knowledge Query and Manipulation Language (KQML)
 - FIPA-ACL (FIPA = Foundation for Intelligent Physical Agents)

Interacting Agents (A2A)

<https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interopability/>

- The A2A protocol defines three core actors:
 - User: The end-user initiating a task.
 - Client agent: The requester that formulates and sends a task on behalf of the user.
 - Remote agent: The recipient agent that performs the task and sends back results.
- The Agent Card
 - The agent's name, description, list of capabilities (skills), input schemas, and authentication requirements.
- A typical A2A interaction follows these steps:
 1. Discovery: Agent A (Client) searches for an agent that can perform a specific sub-task (e.g., "book_flight").
 2. Handshake: Agent A reads Agent B's (Remote) Agent Card and authenticates.
 3. Task Delegation: Agent A sends a JSON-RPC message defining the task.
 4. Execution: Agent B performs the work (potentially using its own MCP tools to access a database or API).
 5. Response: Agent B sends the result (or an error) back to Agent A.

Interacting Agents (real MAS are still not here)

- GenAI agent interaction rapidly growing field
- Still prevalent: manually designed AI workflows, consisting of one (typically) or more agents, and external tool calling
- Actual agentic MAS still in their early stages

LLM vs. RAG vs. Agent

LLM rely solely on its training data and general world knowledge. It guesses likely context and responds accordingly, even if the guess may be wrong.

RAG supplements the LLM with retrieved real-world data from a vector database so the answer is grounded in facts the model wouldn't otherwise know.

Agent goes further by deciding which tool /agent to use—database, discount, or otherwise—based on its assigned role and situation, showing reasoning and control before generating the final response.

<https://www.byhand.ai/p/llm-vs-rag-vs-agent-workbook>

	LLM	RAG	Agent  
Knowledge Use	Internet data	Retrieved data	Tools APIs
Control Logic	Single step	fetch then answer	multi-step reason

“The future of language is about the development of language models...”¹

1

Computerlinguistik | Universität Trier | cl.cldh.uni-trier.de